

ValidAI Risk

Rediseño de arquitectura — Copiloto validador nativo en AWS

Alineación con el stack real de trabajo · Ámbitos: metodológica y calibración

Preparado para Andrea · Agosto 2026

1. Objetivo del rediseño

El MVP actual de ValidAI Risk se construyó sobre un stack de curso (OpenAI, Elasticsearch y Cloud Run en GCP) conectado a canales de oficina genéricos (Microsoft 365, Tavily, GitHub público). Ese montaje sirve para demostrar MCP, pero no refleja el entorno real de validación de modelos de riesgo en el que trabaja Andrea.

El objetivo de este rediseño es **acercar la arquitectura al trabajo real** —Claude sobre Amazon Bedrock, con datos en Athena y modelos en SageMaker— manteniéndola **simple pero realista**, y conservando MCP + LangGraph como capa de orquestación (entregable del Módulo 4). En esta primera versión el copiloto cubre dos de los cuatro ámbitos de validación: **metodológica y calibración**.

2. El cambio de fondo

El equipo de validación revisa cuatro tipos de validación: metodológica, de implementación, periódica y de calibración. Hoy en el trabajo solo la metodológica tiene algo automatizado (un RPA), y se busca extender esa lógica a los demás ámbitos. El marco corporativo usa Azure para seguridad y está implementando GitHub Copilot como asistente de código —una herramienta separada, no un canal del copiloto—. El entorno donde vive la data y los modelos es **AWS: Claude vía Bedrock, Athena para consulta de datos y SageMaker para modelos**.

El rediseño, por tanto, **suelta los canales que no representan el trabajo** (Outlook, calendario, búsqueda web, GitHub público) y reconstruye el núcleo sobre servicios AWS, reproduciendo el rol de un validador: recibe archivos de un caso, revisa que la documentación esté alineada a la normativa (SBS) y reta al modelo con análisis de variables y tests estadísticos.

3. Mapeo de servicios: de GCP/OpenAI a AWS

Componente	Hoy (stack de curso)	Rediseño (stack real AWS)
Modelo (LLM)	GPT-4o-mini (OpenAI)	Claude vía Amazon Bedrock
Embeddings	OpenAI text-embedding-3-*	Titan / Cohere Embeddings en Bedrock
RAG / recuperación	Elasticsearch (BM25+kNN+RRF) + HyDE	Bedrock Knowledge Bases (búsqueda híbrida gestionada)
Vector store	Elasticsearch propio	Amazon OpenSearch Serverless
Servidor MCP	Cloud Run (GCP)	AWS Lambda / Bedrock AgentCore
Datos del modelo	(no integrado)	Amazon Athena (consulta sobre S3)
Cálculo de métricas	local en el notebook	Amazon SageMaker
Archivos del caso	carga manual	Amazon S3 (bucket de entrada)
Memoria de hallazgos	SQLite / PostgreSQL	Amazon DynamoDB

Componente	Hoy (stack de curso)	Rediseño (stack real AWS)
Trazabilidad	LangSmith	LangSmith (se mantiene)
Orquestación	MCP + LangGraph	MCP + LangGraph (se mantiene)

Lo importante: MCP no se pierde. AWS Labs ya publica servidores MCP oficiales para Athena / data processing y para Bedrock AgentCore, así que MCP sigue siendo la capa de herramientas — solo que ahora apunta a AWS.

4. El copiloto como validador: tools por ámbito

En vez de siete tools genéricas, el copiloto expone tools que replican el rol de la validadora. Esta versión activa los ámbitos metodológico y de calibración; los otros dos (implementación y periódica) quedan en el roadmap.

Ámbito	Tool MCP	Qué hace / servicios AWS
Metodológica	revisar_documentacion	RAG sobre los archivos del caso vs. normativa SBS y marco del modelo; reta la selección y justificación de variables. Bedrock KB + OpenSearch, con cita de fuente.
Calibración	calcular_calibracion	Ejecuta los tests del manual (Hosmer-Lemeshow, binomial, aproximación normal, semáforo) sobre la data del modelo. Athena (datos) + SageMaker (cálculo).
Calibración	analizar_variables	Análisis de variables y de conteo (poblaciones, faltantes, distribuciones) vía consultas a Athena.
Transversal	evaluar_hallazgo	Clasifica severidad e impacto de cada hallazgo (se mantiene del MVP).
Transversal	consultar_memoria	Recupera hallazgos validados de casos previos (DynamoDB); human-in-the-loop.

5. Flujo funcional

El caso entra por S3; el agente Claude/Bedrock decide qué tool invocar; llama la tool del ámbito (RAG documental para metodológica, o Athena/SageMaker para calibración); arma los hallazgos con evidencia citada; el validador los revisa (acepta, ajusta o descarta); y los hallazgos aprobados van a memoria (DynamoDB) para alimentar futuras validaciones. Un solo flujo, los mismos ámbitos que se ven a diario, sin los canales de oficina que sobraban.

6. Plan de migración por fases

Fase	Alcance	Resultado
Fase 0	Provisionar en AWS: cuenta/roles, S3, acceso a Bedrock (Claude + embeddings)	Base lista sin tocar la lógica del agente
Fase 1	Migrar el RAG: crear Knowledge Base sobre OpenSearch Serverless, reingesta de la documentación	revisar_documentacion (metodológica) funcionando en AWS
Fase 2	Conectar Athena + SageMaker; implementar calcular_calibracion y analizar_variables	Ámbito de calibración operativo
Fase 3	Redeplegar el servidor MCP en Lambda/AgentCore; apuntar el MCP Client a AWS	Copiloto completo sobre stack real
Fase 4	Ámbitos pendientes: implementación y periódica	Roadmap — extender la misma arquitectura

7. Consideraciones

Embeddings: el servidor MCP actual usa text-embedding-3-small, no el -large que figuraba en el proyecto. Al migrar a Titan/Cohere en Bedrock hay que reindexar; conviene alinear el modelo de embeddings con el índice para evitar degradar la recuperación.

Seguridad corporativa: Azure y GitHub Copilot son el entorno de código del equipo, separado del copiloto. La data sensible se mantiene dentro de AWS (S3/Athena) con anonimización básica en la etapa de preparación; nada del caso sale a servicios externos.

MCP sobre AWS: mantener MCP no es sobre-ingeniería. Es el entregable M4 y permite que el mismo servidor funcione desde Claude Desktop, Cursor u otra app compatible sin reescribir integraciones. La capa interna corre en Lambda/AgentCore y no se ve en la demo (solo Streamlit).

Gobernanza: se conserva el human-in-the-loop. El copiloto propone hallazgos con cita; la validadora aprueba antes de escribir a memoria. No hay auto-mejora normativa — límite de diseño ya definido para Fase 2.

8. Diagramas

Arquitectura AWS del copiloto validador



Workflow funcional

